

Homework 4

- ① a) To meet the given time complexities, we will use a data structure that will be composed of two heaps, a min-heap and a max-heap.

This structure will partition n elements into two halves, a lower half and an upper half.

Min-Heap $\rightarrow$ This heap will store the larger half of the elements, where the smallest element of this larger half can be accessed at the root in $O(1)$ time.

Max-Heap $\rightarrow$ This heap will store the smaller half of the elements, where the largest element of this smaller half can be accessed at the root in $O(1)$ time.

For this structure to work,

- $\rightarrow$ The number of elements in both the heaps must not differ by more than 1.
- $\rightarrow$ Any element in the max-heap with the smaller half must be less than or equal to any element in min-heap with the larger half.

The median of the entire set of elements is located at the root of the larger of the two heaps. If the two heaps are equal then the median is the root of the max-heap with smaller elements. This makes finding the median an $O(1)$ operation.

b) Insert() algorithm

We will add a new number x to the structure and rebalance the heap.

1. Initial Placement
 - If the max-heap is empty or if x is less than or equal to the root of the max-heap, add x to the max-heap.
 - Otherwise, add x to the min-heap.
2. Rebalancing:
 - If the max-heap has more than one element more than the min-heap, pop the root from the max-heap and push it into the min-heap.
 - If the min-heap has more than one element compared to the max-heap, pop the root from the min-heap and push it into the max-heap.

∴ Both initial placement and rebalancing takes $O(\log n)$ time. Total time complexity: $O(\log n)$

Extract-Median() algorithm:

1. Find median

→ Identify the larger heap, its root is the median.

→ If heaps are equal in size, the root of the max-heap is the median.

2. Remove median

→ pop the root from the heap that contains the median.

3. Rebalancing:

→ Perform the same rebalancing step as in Insert() function to ensure the two heaps have size difference of not more than 1.

∴ Finding and removing the median takes $O(\log n)$ time and rebalancing also takes $O(\log n)$ time.

∴ Total time complexity is $O(\log n)$.

(2) The goal is to find the single candidate link $(u, v) \in E$ that provides the greatest reduction in the shortest path latency between source s and a target t .

Instead of repeatedly adding each link and re-running Dijkstra, we will pre-calculate all necessary distances first and then evaluate the impact of each candidate link in constant time.

Algorithm

1. Pre-computation
 - Run Dijkstra's algorithm from s to compute $\text{dist}_s[v]$ - the shortest distance from s to every node v .
 - Run Dijkstra's again from t to compute $\text{dist}_t[v]$, the shortest distance from t to every node v .

2. Evaluating candidate links

- Record the current shortest path length: $\text{orig} = \text{dist}_s[t]$.

- for each candidate link (u, v) in C :

- Compute the new possible path length using the added link, where we assume the new link has 0 latency.

$$\text{new} = \min(\text{dist}_s[u] + \text{dist}_t[v], \text{dist}_s[v] + \text{dist}_t[u])$$

7. Calculate the potential improvement:
 $\text{reduction} = \text{orig} - \text{new}$

8. Track the link that yields the largest reduction.

3. Result:
 - The candidate link with the maximum reduction is the best choice to add to the network.

Runtime Complexity

1. Dijkstra from s : $O((m+n) \log n)$

2. Dijkstra from t : $O((m+n) \log n)$

3. Evaluation of k candidate links: $O(k)$

Total complexity:

$$O((m+n) \log n) + O((m+n) \log n) + O(k)$$

$$= O((m+n) \log n + k)$$

It is not worse than Dijkstra's algorithm as it is dominated by two Dijkstra runs.

- ③ The goal is to find a minimum-cost spanning tree where vertex s is a leaf (degree = 1).

To achieve this, we will split the problem into two parts:

1. Select the cheapest edge incident to s .
2. Compute MST of the subgraph induced by $V' = V - \{s\}$.

Algorithm:

1. Build subgraph: Remove vertex s and all its incident edges from G to form $G' = (V - \{s\}, E')$.
2. Use BFS or DFS to verify G' is connected. If it's not, no valid solution exists.
3. Find the minimum Spanning Tree T' of G' using Kruskal's or Prim's algorithm. Let its cost be C_{MST} .
4. From the original graph G , pick the cheapest edge $e_s = (s, v)$ connecting s to any other vertex. Let its cost be C_s .
5. The final tree $T = T' \cup \{e_s\}$ connects all vertices, satisfies $\deg(s) = 1$, and has total cost $C_{\text{total}} = C_{\text{MST}} + C_s$.

Correctness:

1. T' spans all vertices except s . Adding e_s connects s without creating a cycle, forming a spanning tree.
2. Only one edge connects to s , ensuring $\deg(s) = 1$.
3. Since T' is the MST of G' and e_s is the minimum-cost edge incident to s , no lower-cost valid tree exists.

Complexity:

1. Subgraph construction: $O(n+m)$
2. Connectivity check: $O(n+m)$
3. MST computation: $O(m \log n)$
4. Min edge selection: $O(n)$

Overall complexity: $O(m \log n)$